

BEGINNER-FRIENDLY GUIDE

How the AI Learns

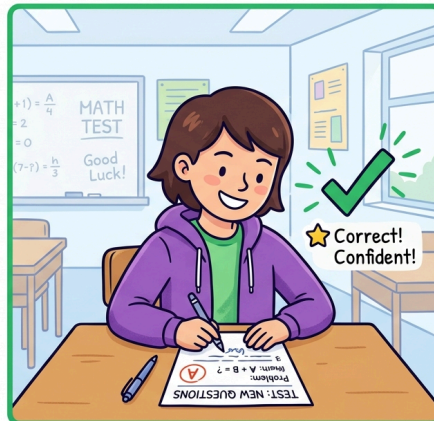
Machine Learning Explained for Everyone

PANEL 1: LEARNING FROM EXAMPLES



LEARNING PHASE

PANEL 2: CONFIDENTLY ANSWERING TEST



PREDICTION PHASE

Project: Smart Indoor Garden — Predictive Hydroponic Optimization

Institution: YSK 2026 — Kisii School

Audience: High School Students & Beginners

Date: August 2026

1. What is Machine Learning?

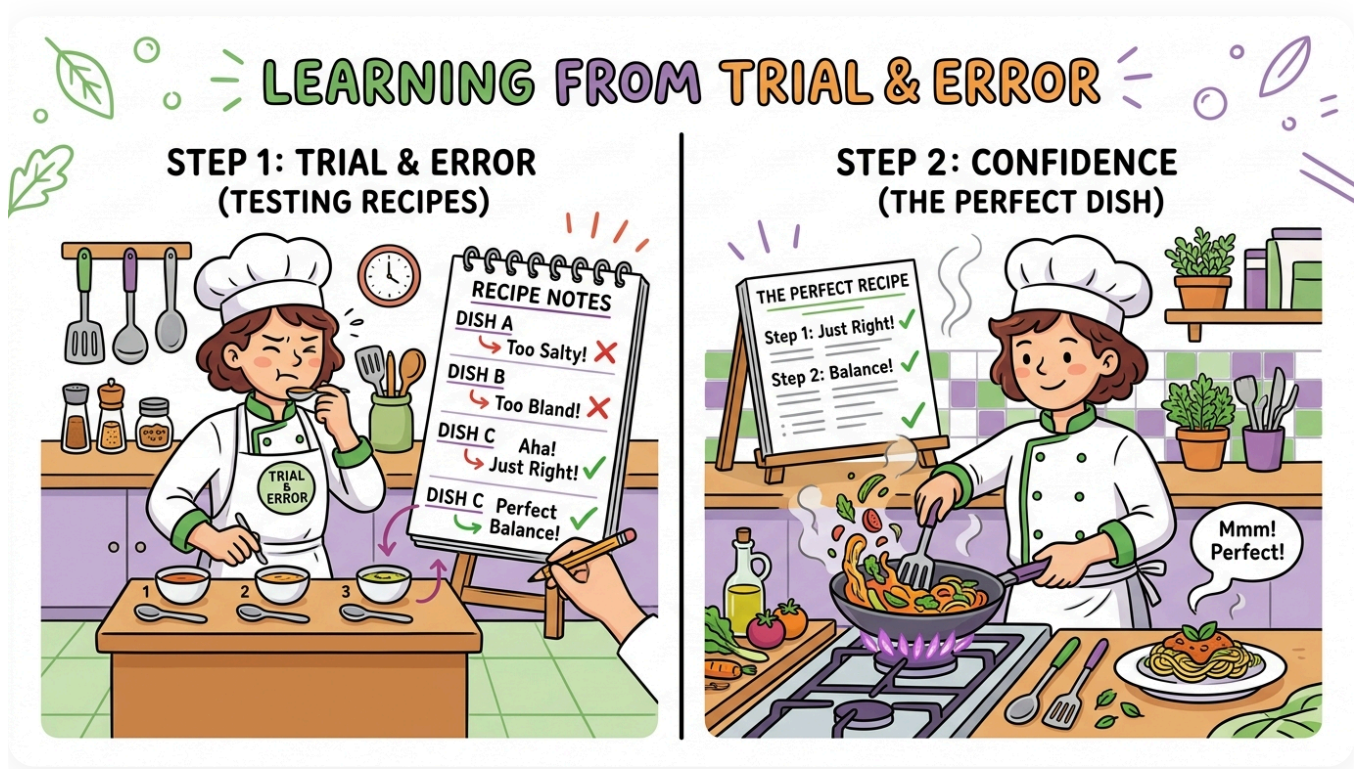
Machine Learning (ML) is simply **teaching a computer to make decisions by showing it lots of examples**, instead of writing step-by-step rules for every situation.

💡 ANALOGY

Think of it like learning to cook:

Imagine you want to learn how much salt to add to different dishes. You could read a book of exact rules for every single food (that's normal programming). **OR** — you could taste 50 different dishes, note how much salt each one had, and eventually you just *know* how much salt a new dish needs. That's machine learning!

The computer "tastes" hundreds of examples and finds the pattern on its own.



Just like a chef learns from tasting many dishes, our AI learns from many sensor readings

How This Applies to Our Garden

Our hydroponic garden needs to decide: **"How long should I run the pump to add nutrients?"**

Instead of writing complicated rules like *"if temperature is above 25 AND pH is below 6.5 AND nutrients are between 200-300, then run pump for exactly 4.7 seconds"* (which would be impossible for every combination) — we let the AI **learn the pattern from examples**.

KEY IDEA

We show the AI many examples of: **"When the garden looked like THIS → a human expert said the pump should run for THIS MANY seconds."** After seeing enough examples, the AI figures out the pattern and can decide on its own for new situations.

The Three Steps of ML in Our Project

1

Collect Examples

A human looks at the sensors and records: "For *these* conditions, the pump should run for *this* long."
This is our **training data**.

2

Train the AI

The computer studies all the examples and finds the **hidden pattern** — the relationship between sensor readings and dosing time.

3

Make Predictions

Now the AI can look at **brand new** sensor readings and predict the correct dosing time — even for situations it has never seen before!

2. The Training Data — Teaching by Example

Before the AI can learn anything, we need to give it **examples to study**. These examples are stored in a simple spreadsheet-like file called **garden_data.csv**.

ANALOGY

Think of it like a study guide for an exam:

Imagine a teacher gives you a sheet with 200 practice problems AND their correct answers. You study the pattern: *"Oh, when the question looks like this, the answer is usually around 5. When it looks like that, the answer is closer to 12."* The CSV file is exactly this — a study guide with problems (sensor readings) and answers (dosing times) for the AI.

What Each Example Looks Like

Every time a human clicks **"Record Calibration"** on the dashboard, one new example is saved. Each example has 5 pieces of information:

What We Measure	What It Means	Example	Think of it as...
 Temperature	How hot the air is near the plants	25.3°C	Like checking the weather
 Humidity	How moist the air is	68%	How "sticky" it feels
 pH Level	Is the water acidic or basic? Plants like it around 6.0	6.42	Like a swimming pool test strip
 Nutrient Level	How much "plant food" is in the water (0–1023)	312	Like checking if your meal has enough vitamins
 Dosing Time	How long the pump should run (set by the human)	5.0 seconds	The correct answer the AI needs to learn

IMPORTANT DISTINCTION

The first 4 measurements (temp, humidity, pH, nutrients) are read **automatically** by the sensors. The **dosing time is the only thing the human sets** — and that's the "answer" the AI is trying to learn to predict on its own.

Why More Examples = Better AI



ANALOGY

Think about it this way:

If you only studied **3 practice problems** before an exam, you'd probably guess wrong on most new questions. But if you studied **200 practice problems** covering all sorts of scenarios, you'd be much more confident. The AI works the same way — more diverse examples make it smarter and more accurate.

5

Minimum to train

Like studying the night before — it sort of works but makes lots of mistakes

50

Good dataset

Like studying all week — the AI gets most predictions right

200

Excellent dataset

Like studying all term — the AI becomes really accurate and reliable

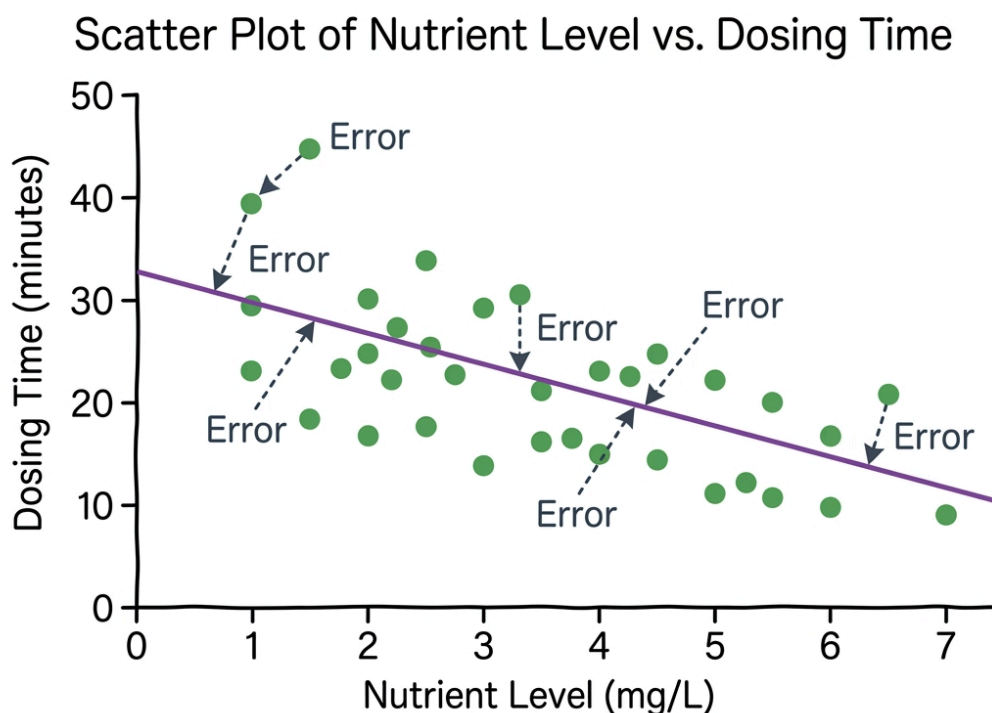
3. Method 1: Linear Regression — Drawing the Best Line

The simplest way our AI can learn is called **Linear Regression**. Don't let the fancy name scare you — it's really just **drawing the best straight line through a cloud of dots**.

💡 ANALOGY

Imagine you're plotting grades vs study hours on graph paper:

You ask 30 classmates: "How many hours did you study?" and "What grade did you get?" You plot each person as a dot. Then you take a ruler and draw the **one straight line that passes as close as possible to ALL the dots**. Now, if a new student says "I studied 5 hours," you can use your line to predict their grade. That's Linear Regression!



The purple line is the "best fit" — it gets as close to all the green dots as possible

How It Works in Our Garden

The computer draws a "line" (actually a formula) that connects sensor readings to dosing time. The formula looks like this:

$$\begin{aligned} \text{Dosing Time} = & \text{starting point} \\ & + (\text{weight}_1 \times \text{temperature}) \\ & + (\text{weight}_2 \times \text{humidity}) \\ & + (\text{weight}_3 \times \text{pH}) \\ & + (\text{weight}_4 \times \text{nutrients}) \end{aligned}$$

The "weights" are just numbers that tell the AI **how important** each sensor is and **in which direction** it pushes the dosing time.

ANALOGY

Think of the weights like volume knobs on a music mixer:

Each sensor gets its own "volume knob." Training is the process of **turning each knob up or down** until the output sounds right (matches the correct answers). A weight of +3.5 means "this sensor pushes dosing time UP a lot." A weight of -1.2 means "this sensor pushes dosing time DOWN a little."

What "Training" Actually Does

Training is the computer trying **millions of different weight combinations** until it finds the set of weights that gives the smallest total error across ALL examples. It's like a music producer tweaking knobs until the song sounds perfect.

IN PLAIN ENGLISH

Linear Regression says: **"I believe the answer is a simple recipe — just multiply each sensor reading by some number, add them up, and that's my prediction."** The computer finds the best numbers to multiply by.

LIMITATION

A straight line can only capture **simple, proportional relationships**. If the real pattern is curved or complex (e.g., pH matters way more when nutrients are also low), Linear

Regression can't capture that. For complex patterns, we need the Neural Network — explained next!

4. Method 2: Neural Network (MLP) — A Mini Brain

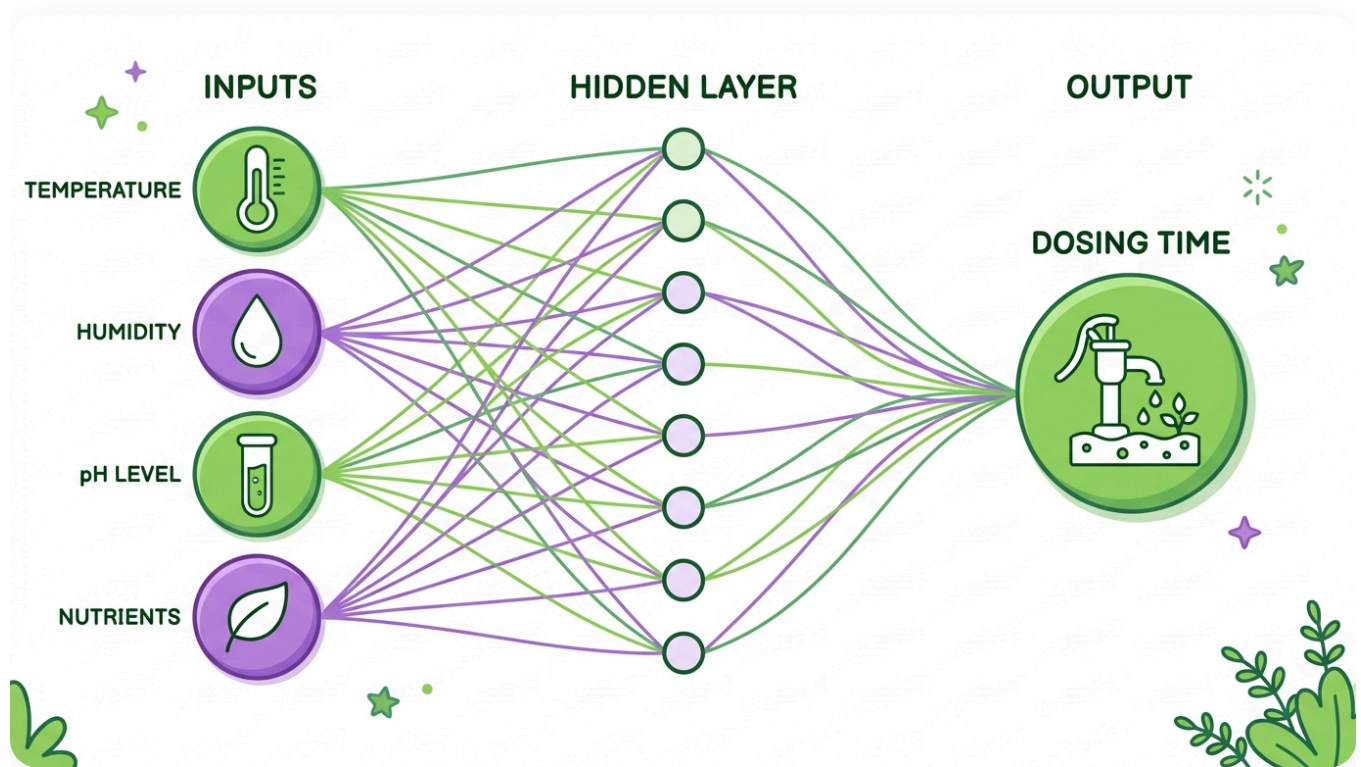
The second method is called a **Multi-Layer Perceptron (MLP)** — a type of neural network. Think of it as a smarter, more flexible version of the line-drawing method.

ANALOGY

Imagine a team of 8 advisors helping a manager make a decision:

The manager needs to decide how long to run the pump. Instead of looking at the sensors directly, they ask **8 expert advisors**. Each advisor looks at ALL four sensors, thinks about it in their own unique way, and gives a recommendation. The manager then listens to all 8 advisors and combines their opinions into one final answer.

Some advisors are "temperature experts," some care more about pH — each one weighs the inputs differently. Together, they can understand **complex patterns** that no single person could catch alone.



Our neural network: 4 inputs → 8 "advisors" (hidden neurons) → 1 output (dosing time)

The Three Layers Explained

IN

Input Layer — "The Questions"

The 4 sensor readings come in: Temperature, Humidity, pH, and Nutrient Level. These are like the "exam questions" being passed to the advisors. Each value is scaled to be between 0 and 1 (so they're all on the same playing field).

H

Hidden Layer — "The 8 Advisors" (This is where the magic happens!)

Each of the 8 neurons (advisors) receives ALL 4 inputs, multiplies each by its own personal weight, adds them up, and decides: **"Should I activate or stay quiet?"** (This yes/no decision is called ReLU — if the sum is negative, the advisor says nothing. If positive, they pass their value forward.)

It's called "hidden" because you never see this layer directly — it works behind the scenes.

OUT

Output Layer — "The Final Decision"

The manager neuron listens to all 8 advisors, weighs their opinions, and produces **one final number**: the predicted dosing time in seconds.

Why Is This Better Than a Straight Line?



ANALOGY

Think of it like directions to school:

Linear Regression can only give directions using one straight road: "Go north for 2km." But the Neural Network can give complex directions: "Go north, then turn left at the roundabout, take the third exit, then curve right." It can follow **twists and turns** in the data that a straight line would miss completely.

FUN FACT

Our neural network has **49 weights** in total (8 advisors $\times$ 4 inputs + 8 biases + 8 output weights + 1 output bias = 49 numbers). During training, the computer adjusts all 49 knobs simultaneously until the predictions are as close to the correct answers as possible. The big

AI models like ChatGPT have **billions** of these weights — ours is tiny but perfectly suited for our garden!

5. The Training Process — How the AI "Studies"

Training is the step where the computer actually **learns from the data**. Here's exactly what happens behind the scenes when you press "Execute Machine Learning Pipeline":

ANALOGY

Imagine learning to throw darts blindfolded:

You throw a dart (guess an answer). A friend tells you "You were 3 inches too far left." You adjust and throw again. "Now you're 1 inch too high." You adjust again. After hundreds of throws with feedback, you're hitting near the bullseye almost every time — even blindfolded. This is exactly how the AI trains: **guess, check the error, adjust, repeat**.

Step-by-Step: What the Computer Does

1

Load the Examples

The computer reads all 200 rows from the CSV file. Each row becomes one practice problem. The sensor readings are the "question" and the dosing time is the "correct answer."

2

Normalize the Numbers (Put Everyone on the Same Scale)

Temperature goes up to 50, but humidity goes up to 100, and nutrients go up to 1023. That's unfair — bigger numbers would dominate smaller ones. So we **shrink everything to a scale of 0 to 1** by dividing each value by its maximum.

Temperature $25^{\circ}\text{C} \rightarrow 25 \div 50 = 0.50$ Humidity $68\% \rightarrow 68 \div 100 = 0.68$

Now all inputs are on equal footing — like converting different currencies into one standard currency.

3

Start With Random Weights (Random Guesses)

The computer assigns random numbers to all the weights. At this point, its predictions are terrible — like a student guessing randomly on a test. That's okay — this is just the starting point.

4

Make Predictions and Measure the Error

The computer runs every example through the formula and compares its prediction to the correct answer. The difference is the **error**. It adds up all the errors across all 200 examples to get one big "total error score."

5

Adjust the Weights to Reduce Error (The Learning!)

The computer nudges each weight slightly — some up, some down — trying to make the total error **smaller**. It uses math (calculus) to figure out which direction each weight should move. This is repeated **thousands of times** (up to 5,000 rounds for our MLP).

6

Stop When It's Good Enough

Eventually, the errors get so small that more adjustments don't help much. The weights are now "learned" — they capture the pattern in the data. Training is done!

IN SUMMARY

Training is just: **Guess** → **Check Error** → **Adjust** → **Repeat (thousands of times)**. Each round, the predictions get a little more accurate. By the end, the AI has "learned" the relationship between sensor readings and dosing times.

6. How Do We Know If the AI Learned Well?

After training, we need to check: **"Did the AI actually learn, or is it still guessing?"**

We use two report card grades to measure this.

Grade 1: R^2 Score — "How Much of the Pattern Did It Capture?"



ANALOGY

Think of R^2 like a percentage grade on a test:

An R^2 of **0.85** means the AI understood **85% of the pattern** in the data. The remaining 15% is noise or patterns too complex for the model to catch. An R^2 of **1.0** would be a perfect 100% — the AI nailed every single prediction exactly.

Poor (20%)

0.20

The AI barely learned anything. It's almost just guessing randomly. Needs more data or a better model.

Decent (65%)

0.65

The AI learned the basic pattern but misses some nuances. Acceptable for simple tasks.

Good (85%)

0.85

The AI learned most of the pattern. Its predictions are usually close to correct. This is our target!

Excellent (95%)

0.95

The AI captured almost all the pattern. Predictions are very accurate and reliable.

Grade 2: MAE — "How Far Off Are the Predictions On Average?"



ANALOGY

Think of MAE like the average number of points you miss on a quiz:

If the MAE is **1.2 seconds**, it means the AI's predictions are off by about 1.2 seconds on average. So if the correct answer was "run the pump for 5.0 seconds," the AI might say 3.8 or 6.2 seconds. That's pretty close! If the MAE was 8.0 seconds, the AI would be wildly inaccurate.

MAE Value	What It Means	Is It Good?
Under 1.0s	Predictions are off by less than 1 second	☀️ Excellent — very precise
1.0 — 2.5s	Predictions are off by 1–2.5 seconds	✅ Good — close enough for the garden
2.5 — 5.0s	Predictions are off by a noticeable amount	⚠️ Needs more data or better model
Above 5.0s	Predictions are way off — almost useless	❌ Poor — retrain with more examples

The Actual vs. Predicted Table

After training, the Garden-AI Studio shows you a table comparing what the AI *predicted* versus the *actual correct answer* for every example. This lets you visually see how close the AI's guesses are.

Example #	Actual Dosing (correct)	AI Predicted	Error (how far off)
1	5.0 seconds	4.8 seconds	0.2s ✅
2	8.5 seconds	7.9 seconds	0.6s ✅
3	2.0 seconds	3.1 seconds	1.1s ⚠️
4	12.0 seconds	11.3 seconds	0.7s ✅

SIMPLE RULE

Higher R^2 = better. It means the AI understood more of the pattern.

Lower MAE = better. It means the AI's predictions are closer to the correct answers.

7. Deployment — Putting the AI Brain Into the Garden

After training, the AI's "knowledge" is just **a small list of numbers** — the weights it learned. We save these numbers in a tiny file and upload it to the garden's microcontroller.

ANALOGY

Think of it like a cheat sheet:

After studying 200 practice problems, a student writes down the key formulas on a small index card. They don't need the textbook anymore — just the card with the important numbers. That index card is our `model_params.json` file — a tiny file containing only the learned weights.

What's Inside `model_params.json`?

It's just a text file with numbers. For the Linear Regression model, it looks like this:

```
// This is the AI's "cheat sheet" — just 5 numbers!
{
  "model_type": "linear",
  "w_dosing": [
    8.2,      ← starting point (if all sensors were zero)
    -1.5,     ← temperature weight (hotter → slightly less dosing)
    -0.3,     ← humidity weight (more humid → slightly less dosing)
    +2.8,     ← pH weight (higher pH → MORE dosing needed)
    -4.1      ← nutrient weight (more nutrients → LESS dosing)
  ]
}
```

That's it! The entire "intelligence" of a Linear Regression model is **just 5 numbers**. The MLP neural network has 49 numbers — still incredibly small.

How the ESP-01 Uses the Model

1

Upload the Weights

You drag-drop `model_params.json` onto the dashboard. The file is saved to the ESP-01's flash memory (like saving to a USB drive).

2

Switch to AI-Edge Mode

Click "AI-Edge" on the dashboard. Now the microcontroller runs the learned formula every 15 seconds using **live sensor data**.

3

Real-Time Predictions

The ESP-01 reads the current sensors, plugs them into the formula with the learned weights, and gets a prediction. If it says "dose for 4.2 seconds," it tells the Arduino to run the pump for exactly 4.2 seconds. **No internet needed — it all runs locally on a chip smaller than a coin!**

THIS IS CALLED "EDGE AI"

When an AI model runs directly on a tiny device (like our ESP-01) instead of on a big cloud server, it's called **Edge AI**. The "edge" means the outermost point of a network — the actual device in the field. Our garden doesn't need WiFi or the internet to make predictions. The trained brain lives right inside the garden itself!

The Self-Learning Loop

Here's the coolest part: our system can **keep learning and improving itself!** A background program on a laptop connected to the garden automatically:

1. Downloads new calibration data from the garden every 10 seconds
2. Retrains the AI model with the new data
3. Only deploys the updated model if it's **better than the previous one** (higher R^2 score)
4. The garden gets smarter over time — automatically, without human intervention!

THE BIG PICTURE

Data → Training → Weights → Deployment → Predictions → More Data → Retrain → Better Weights → Better Predictions. It's a **virtuous cycle** — the AI keeps getting smarter the more the garden runs.